Custom XML → public pairs + rate: partner migration plan

We are **sunsetting server-side custom XML generation**. Personalized **fees and limits stay in place**. If you currently poll a ready-made XML file from us, you will need to build and host that XML yourself.

This document is for B2B partners who consume:

- `GET /api/v2/rates/public/xml-v2-custom?path=…`
- or `GET /api/v2/rates/public/xml-v2?referrerId=…`

Those URLs will stop returning a generated feed. The public pair list and public rate endpoints remain.

What changes vs what stays

	After sunset
Ready-made custom XML file from Quickex	**Gone.** You generate `<rates>` yourself.
Pair catalog for your affiliate	**Stays.** `GET /api/v2/pairs/all?referrerId={aff_id}`
Your fees, min/max, network-fee flags, pair multipliers	**Stay.** Applied automatically when you pass `referrerId` into the rate endpoint.
Main public XML (`/api/v2/rates/public/xml-v2` without `referrerId`)	Unchanged. That is the default BestChange feed, not a partner feed.

You do **not** re-implement fee formulas. Pass your affiliate id (`referrerId`) on every rate call; we apply your custom XML fee and limit logic on our side.

Target architecture

```
1. GET /api/v2/pairs/all?referrerId={aff_id}     → full pair list
(paginated)
2. For each active pair:
     GET /api/v2/rates/public/one?…&referrerId={aff_id}
3. Map JSON → BestChange XML <item>
4. Serve the file from your side (BestChange / aggregators poll you)
```

Refresh on a schedule (for example every 1–5 minutes). Do not request the full catalog on every visitor hit.

Step 1. Load your pair list

```
GET https://quickex.io/api/v2/pairs/all?referrerId={YOUR_AFF_ID}
```

- No authentication.
- `referrerId` is your affiliate id (`aff_…`). Without it you get the **global** catalog, not your partner pairs.
- Do **not** use `/api/v2/pairs/public` — that is popular pairs only, not affiliate-scoped.

Pagination

Default `limit` is `1000` and is hard-capped at `1000`. If `total` is larger than the page, request the next page with `offset`.

```
# page 1
curl -sS "https://quickex.io/api/v2/pairs/all?
referrerId=aff_YOUR_ID&limit=1000&offset=0"

# page 2
curl -sS "https://quickex.io/api/v2/pairs/all?
referrerId=aff_YOUR_ID&limit=1000&offset=1000"
```

Loop while `offset + len(pairs) < int(total)`.

Optional filters (`instrumentFromCurrencyTitle`, `instrumentFromNetworkTitle`, `instrumentToCurrencyTitle`, `instrumentToNetworkTitle`) are exact-match AND. For a full XML dump, omit them.

Response


```
{
  "total": "8",
  "pairs": [
    {
      "instrumentFrom": {
        "currencyTitle": "BTC",
        "networkTitle": "BTC",
        "slug": "btc",
        "precisionDecimals": 8,
        "reservesAmount": "12.5",
        "bestChangeCurrencyName": "BTC"
      },
      "instrumentTo": {
        "currencyTitle": "USDT",
        "networkTitle": "TRX",
        "slug": "usdt-trx",
        "precisionDecimals": 6,
        "reservesAmount": "250000",
        "bestChangeCurrencyName": "USDTTRC20"
```

```
      },
      "isActive": true,
      "minAmount": "0.001",
      "maxAmount": "5"
    }
  ]
}
```

Field	Use in XML
`instrumentFrom` / `instrumentTo` `currencyTitle` + `networkTitle`	Rate request identifiers
`bestChangeCurrencyName`	`<from>` / `<to>` (BestChange ticker)
`instrumentTo.reservesAmount`	`<amount>` (reserve)
`minAmount` / `maxAmount`	`<minamount>` / `<maxamount>`
`isActive`	Skip the pair if `false`

Unknown `referrerId` → **404** with `{ "total": "0", "pairs": [] }`. Confirm the aff id with your account manager.

Step 2. Quote each pair

```
GET https://quickex.io/api/v2/rates/public/one
```

`/api/v1/rates/public/one` is the same handler (rewritten to v2). Prefer **v2**.

Required query params

Param	From pair list
`instrumentFromCurrencyTitle`	`instrumentFrom.currencyTitle`
`instrumentFromNetworkTitle`	`instrumentFrom.networkTitle`
`instrumentToCurrencyTitle`	`instrumentTo.currencyTitle`
`instrumentToNetworkTitle`	`instrumentTo.networkTitle`
`referrerId`	your aff id (same as in step 1)

Recommended query params

Param	Value
`rateMode`	`FLOATING` (default if omitted; XML `param` is floating)

Param	Value
`claimedDepositAmount`	pair `minAmount` when it is > 0, so the quote sits on your XML min

Example:

```
curl -sS "https://quickex.io/api/v2/rates/public/one?\
instrumentFromCurrencyTitle=BTC&\
instrumentFromNetworkTitle=BTC&\
instrumentToCurrencyTitle=USDT&\
instrumentToNetworkTitle=TRX&\
referrerId=aff_YOUR_ID&\
rateMode=FLOATING&\
claimedDepositAmount=0.001"
```

Passing `referrerId`:

- applies **your** platform fee, network-fee flag, and pair multiplier;
- enforces **your** min/max;
- rejects pairs that are not in your custom XML catalog (`422`, pair not active for affiliate).

If you omit `referrerId`, you get the default public rate — **not** your partner XML rate.

Rate fields you need

JSON	XML
`quotesWithoutNetworkFee.sellQuote.baseValue`	`<in>`
`quotesWithoutNetworkFee.sellQuote.quoteValue`	`<out>`
`finalNetworkFeeAmount`	`<tofee>`

`quotesWithoutNetworkFee` is already fee-adjusted for your affiliate (platform fee / multiplier). Network fee is separate in `<tofee>`, same as our previous custom XML.

Step 3. Build BestChange XML

Emit `Content-Type: text/xml` (or `application/xml`). Structure:

```
<rates>
  <item>
    <from>BTC</from>
    <to>USDTTRC20</to>
    <in>1</in>
    <out>109532.12</out>
    <amount>250000</amount>
    <minamount>0.001</minamount>
    <maxamount>5</maxamount>
```

```
    <tofee>1.5</tofee>
    <param>floating, language</param>
  </item>
</rates>
```

Field mapping

XML	Source
`from`	`pair.instrumentFrom.bestChangeCurrencyName`. If empty, `pair.instrumentFrom.slug` uppercased.
`to`	Same for `instrumentTo`.
`in`	`rate.quotesWithoutNetworkFee.sellQuote.baseValue`
`out`	`rate.quotesWithoutNetworkFee.sellQuote.quoteValue`
`amount`	`pair.instrumentTo.reservesAmount`
`minamount`	`pair.minAmount`
`maxamount`	`pair.maxAmount`
`tofee`	`rate.finalNetworkFeeAmount`
`param`	`floating, language`

Skip the pair when:

- `isActive` is `false`;
- `bestChangeCurrencyName` is empty on from or to **and** you only publish BestChange tickers;
- the rate call fails (no liquidity, inactive, amount out of range).

Escape XML special characters in text nodes (`&`, `<`, `>`). Prefer a real XML encoder, not string concatenation.

Throughput, concurrency, rate limits

Anonymous limits (per IP, sliding window):

Endpoint	Anon	With API key (`x-api-public-key` or `x-api-key`)
`GET /api/v2/pairs/all`	5 / 60s	60 / 60s
`GET /api/v2/rates/public/one`	120 / 60s	600 / 60s

Ask your account manager for a public API key if you quote hundreds of pairs per cycle.

Practical pattern:

1. Fetch **all pages** of `/pairs/all` once per cycle (usually 1–2 calls).
2. Quote pairs with a **small worker pool** (for example 8–15 concurrent `rates/public/one` calls). Do not fire one request per pair in parallel unbounded.
3. On `429`, back off and retry that pair.
4. Cache the assembled XML and serve it from your origin; aggregators should hit **you**, not our rate API per poll.

A catalog of ~500 pairs at 15 concurrent requests finishes in well under a minute with a key.

Reference loop (pseudocode)

```
import math, time, xml.etree.ElementTree as ET
import requests

BASE = "https://quickex.io/api/v2"
AFF_ID = "aff_YOUR_ID"
HEADERS = {"x-api-public-key": "YOUR_KEY"}  # optional but recommended
SESSION = requests.Session()
SESSION.headers.update(HEADERS)

def fetch_all_pairs():
    pairs, offset, limit = [], 0, 1000
    while True:
        r = SESSION.get(f"{BASE}/pairs/all", params={
            "referrerId": AFF_ID, "limit": limit, "offset": offset,
        }, timeout=30)
        r.raise_for_status()
        body = r.json()
        page = body["pairs"]
        pairs.extend(page)
        total = int(body["total"])
        offset += len(page)
        if offset >= total or not page:
            return pairs

def quote(pair):
    params = {
        "instrumentFromCurrencyTitle": pair["instrumentFrom"]
["currencyTitle"],
        "instrumentFromNetworkTitle": pair["instrumentFrom"]
["networkTitle"],
        "instrumentToCurrencyTitle": pair["instrumentTo"]["currencyTitle"],
        "instrumentToNetworkTitle": pair["instrumentTo"]["networkTitle"],
        "referrerId": AFF_ID,
        "rateMode": "FLOATING",
    }
    min_amt = pair.get("minAmount") or "0"
    if float(min_amt) > 0:
        params["claimedDepositAmount"] = min_amt
    r = SESSION.get(f"{BASE}/rates/public/one", params=params, timeout=15)
    if r.status_code != 200:
```

```
            return None
        return r.json()

    def bc_name(instrument):
        return instrument.get("bestChangeCurrencyName") or
    (instrument.get("slug") or "").upper()

    def build_xml(pairs):
        root = ET.Element("rates")
        for pair in pairs:
            if not pair.get("isActive"):
                continue
            rate = quote(pair)
            if not rate:
                continue
            q = (rate.get("quotesWithoutNetworkFee") or {}).get("sellQuote") or
    {}
            if not q.get("baseValue") or not q.get("quoteValue"):
                continue
            item = ET.SubElement(root, "item")
            ET.SubElement(item, "from").text = bc_name(pair["instrumentFrom"])
            ET.SubElement(item, "to").text = bc_name(pair["instrumentTo"])
            ET.SubElement(item, "in").text = str(q["baseValue"])
            ET.SubElement(item, "out").text = str(q["quoteValue"])
            ET.SubElement(item, "amount").text =
    pair["instrumentTo"].get("reservesAmount") or "0"
            ET.SubElement(item, "minamount").text = pair.get("minAmount") or
    "0"
            ET.SubElement(item, "maxamount").text = pair.get("maxAmount") or
    "0"
            ET.SubElement(item, "tofee").text =
    str(rate.get("finalNetworkFeeAmount") or "0")
            ET.SubElement(item, "param").text = "floating, language"
            time.sleep(0)  # replace with a worker pool + 429 backoff in
    production
        return ET.tostring(root, encoding="unicode")
```

Production: paginate, bound concurrency, retry 429/5xx, skip failed pairs, atomically replace the published file.
